Software Requirements Specification for Software Engineering: subtitle describing software

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

October 19, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Stakeholders - ITS STAKEHOLDER TIME BOI . . . 3
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 4

2 System Overview **4**
- 2.1 Problem Statement . . . 4
- 2.2 Product Vision . . . 5
- 2.3 System Context . . . 5
- 2.4 User Characteristics . . . 7
- 2.5 System Constraints . . . 7

3 Stakeholder Requirements **8**

4 User Personas **9**
- 4.1 Persona 1 . . . 9
- 4.2 Persona 2 . . . 9
- 4.3 Persona 3 . . . 10
- 4.4 Persona 3 . . . 11

5 Use Cases **11**
- 5.1 UC1: Submit Course Outline . . . 11
- 5.2 UC2: Review Extracted Course Information . . . 12
- 5.3 UC3: Review Course Outline Against McMaster and CAS Policies . . . 13
- 5.4 UC4: Review Course Outline Against CEAB Criteria . . . 14
- 5.5 UC5: Review Course Outline Findings . . . 14
- 5.6 UC6: Submit Tentative Assessment Dates . . . 15
- 5.7 UC7: Analyze Assessment Schedule . . . 16
- 5.8 UC8: Generate Scheduling Recommendations . . . 17
- 5.9 UC9: Review and Decide on Scheduling Recommendations . . . 18
- 5.10 UC10: Update Assessment Dates and Re-analyze Schedule . . . 18

6 Functional Requirements **19**

7 Data Requirements **21**
- 7.1 Major Data Entities . . . 21
- 7.2 Data Dictionary . . . 22
- 7.3 Privacy and Data Retention . . . 23

8 Non-Functional Requirements **23**

9 AI and Machine Learning Requirements **26**
9.1 Model Inputs . . . 26
9.2 Model Outputs . . . 26
9.3 Evaluation Metrics . . . 27
9.4 Known Limitations . . . 27

10 Requirements Traceability Matrices **28**

11 Rationale **29**

12 Likely Changes **29**

13 Unlikely Changes **29**

14 Appendices **30**
14.1 Appendix A: Additional Materials . . . 30
14.2 Appendix B: Development Plan . . . 30
14.3 Appendix C: Values of Auxiliary Constants . . . 30
14.4 Appendix D: Generative AI Disclosure . . . 30
14.5 Appendix E: Reflection . . . 30

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[All capstone SRS documents should have a reflection appendix. —TPLT]

[This SRS template is based on the IEEE template, with input from co-pilot and S. Smith. It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. —TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

1 Introduction

[Explain what system is being specified. State purpose, users, and project goals. —TPLT]

[Introduce the name of your software as Software Engineering. Using Software Engineering as a command lets you change the name if necessary and have the change propagate to all documents. —TPLT]

[Include a reference to the template this work is based on. **?** justifies the SRS structure and **?** provides the broader life-cycle context that underlines modern requirements engineering practice. —TPLT]

1.1 Purpose

The CAS (Computing and Software) Department currently manages course outlines and assessment schedules on a course-by-course basis. Course instructors set their deadlines independently, so students in the same program and year can face busy weeks of midterms and deadlines with no department-level visibility into them. CAS administrative staff must also manually check each outline against McMaster course outline policy and the Canadian Engineering Accreditation Board (CEAB) requirements, and follow up with instructors when information is missing.

The CAS Deadline Management System is intended to make assessment dates visible across courses and to identify policy and accreditation gaps early. It reduces the manual effort of checking outlines against these requirements and helps avoid weeks with heavily clustered assessments. Instructors and administrators remain in the loop and responsible for every decision. The system only flags issues and makes suggestions, and never changes a course outline or assessment date on its own.

1.2 Scope

The CAS Deadline Management System aims to improve course outline review and assessment scheduling within the CAS Department at McMaster University. The system addresses the lack of department-wide visibility into assessment dates and the manual effort required to verify course outlines against university policies and CEAB requirements. The system will allow CAS administrators to upload course outlines, extract and store relevant course and assessment information, identify missing information and potential policy or accreditation gaps, and visualize assessment schedules across courses. It will also identify potential assessment conflicts and suggest alternative dates to help distribute assessments more evenly throughout the academic term. The system will support reviewing and exporting extracted information to facilitate administrative coordination.

The primary users are CAS administrators and course instructors, while students may benefit indirectly from improved assessment scheduling. The system is intended to reduce administrative workload, improve policy compliance, and support more balanced assessment schedules.

The system will provide recommendations for review rather than make decisions independently. It will not automatically modify course outlines, change assessment dates, or override instructor decisions. Instructors retain authority over their course schedules, and administrators remain responsible for reviewing policy compliance and accreditation requirements. The system's recommendations will be based on the available course outline data, timetable information, and defined policies; it will not guarantee that all scheduling conflicts or compliance issues are identified.

1.3 Stakeholders - ITS STAKEHOLDER TIME BOI

The CAS Midterm and Assignment Optimization System is intended to support the Computing and Software (CAS) department at McMaster University in reviewing course outlines provided by instructors and coordinating assessment schedules. The stakeholders include individuals who directly use the system, individuals affected by its recommendations, and organizations responsible for ensuring policy requirements and accreditation.

Primary Stakeholders:

- CAS Administration Team (Customer)
- CAS Department Professors and Instructors (End Users)
- Capstone Project Team 19 (Development Team)

Secondary Stakeholders:

- Software Engineering Students (Indirectly affected by assessment scheduling recommendations)
- McMaster University (Affected by the system's support for course coordination and policy compliance)

Tertiary Stakeholders:

- Canadian Engineering Accreditation Board (CEAB)
- Future CAS Administrators and Instructors

1.4 Definitions and Acronyms

Term	Definition
CEAB	Canadian Engineering Accreditation Board, the organization responsible for accrediting undergraduate engineering programs in Canada.
LLM	Large Language Model, an AI model trained on large amounts of text to understand and generate human language.
SRS	Software Requirements Specification, a document that describes a software system's requirements, including its expected functionality and constraints.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended readers of this Software Requirements Specification include project supervisors, CAS administrators, software developers, testers, and future maintainers. Technical readers should have a foundational understanding of software development, web applications, databases, and software testing. Familiarity with PDF processing and large language models (LLMs) is beneficial but not required. CAS administrators and project supervisors should be familiar with academic scheduling, course outlines, and relevant university policies, including CEAB requirements. This document is intended to communicate system requirements clearly to both technical and non-technical readers.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

The CAS Department at McMaster University currently manages course outlines and assessment schedules independently across courses. Without a centralized view of assessment dates, students in the same program and academic year may face multiple midterms and assignment deadlines within short periods. CAS administrative staff must also manually review course outlines for compliance with university policies and CEAB requirements. Missing or incomplete information, such as midterm dates listed as "TBD," creates additional work and requires repeated communication with instructors.

These challenges contribute to uneven student workloads, increased academic stress, and significant administrative overhead. Furthermore, instructors retain authority over their course schedules, limiting administrators' ability to directly resolve scheduling conflicts.

The CAS Deadline Management System aims to improve this process by centralizing assessment information, identifying potential scheduling conflicts and policy compliance gaps, and suggesting alternative assessment dates. This will help administrators coordinate schedules more efficiently, reduce manual review efforts, and support more balanced student workloads while preserving instructors' authority over their courses.

2.2 Product Vision

The CAS Deadline Management System will provide the CAS Department with a trusted, department-wide foundation for collaborative assessment planning. It will turn course-outline information into a consistent, reviewable source of scheduling knowledge that helps instructors and administrators make informed decisions together. The system is intended to improve coordination and transparency over time, while keeping final responsibility for course schedules and policy interpretation with the appropriate human decision-makers.

2.3 System Context

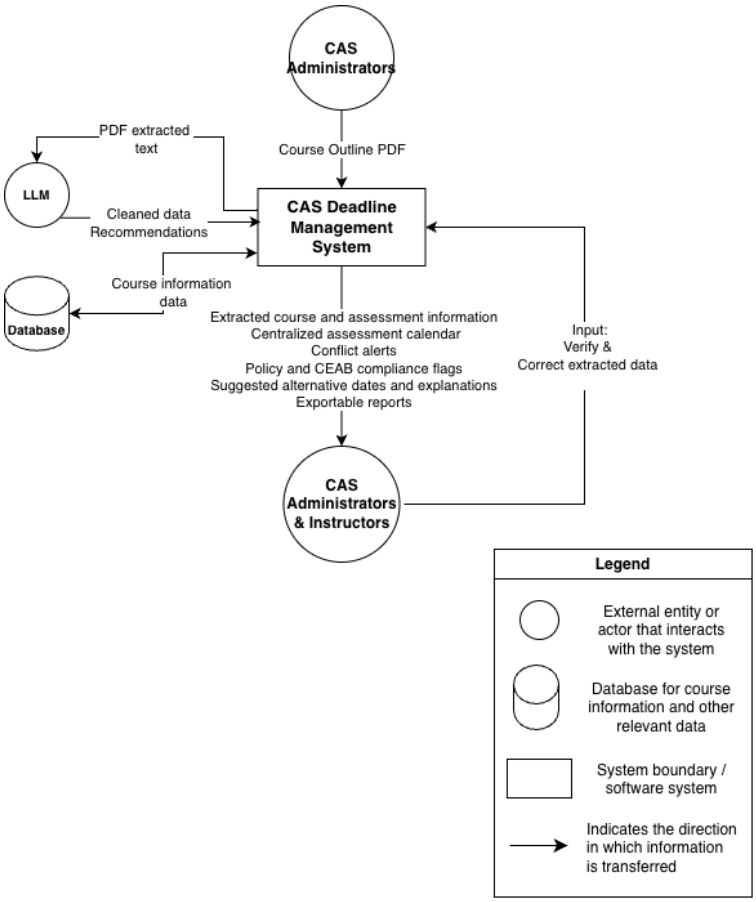


Figure 1: System Context

The CAS Deadline Management System is a web-based application intended to support course outline review, assessment scheduling, and policy compliance within the CAS Department at McMaster University. CAS administrators and course instructors interact with the system to upload course outlines, review extracted information, view assessment schedules, and evaluate potential scheduling conflicts and compliance issues. Figure 1 summarizes

these interactions by showing the system boundary and its relationships with instructors, CAS administrators, the LLM, and the database.

The system processes course outlines and relevant academic scheduling information to extract assessment details, identify missing or potentially non-compliant information, and recommend alternative assessment dates. Results are presented to users for review, while instructors and authorized administrators retain responsibility for final scheduling and compliance decisions.

The system is intended for academic administration and planning rather than mission-critical or safety-critical applications. Since extracted information and automated recommendations may be incomplete or inaccurate, users must review flagged issues and verify important results before making decisions.

- **User Responsibilities:**
  - Provide course outlines and relevant academic scheduling information.
  - Ensure submitted documents are appropriate for review and that the information provided is sufficiently complete and accurate.
  - Review extracted course and assessment information and correct errors where supported.
  - Evaluate flagged scheduling conflicts, missing information, and potential policy compliance issues.
  - Decide whether to accept or reject suggested assessment dates and determine any final changes to course schedules.
- **Software Engineering Responsibilities:**
  - Accept and process supported course outline documents and scheduling information.
  - Extract and organize relevant course and assessment details for review.
  - Validate extracted data for missing, inconsistent, or incorrectly formatted values where possible.
  - Present assessment schedules and identify potential workload conflicts.
  - Flag potential gaps in course outline requirements and applicable accreditation criteria.
  - Generate alternative assessment date recommendations with supporting explanations.
  - Present results and warnings to users without automatically changing course outlines or assessment schedules.

- **External System Responsibilities:**
  - **LLM:** Process course outline content to support information extraction and, where applicable, the generation of explanations for scheduling recommendations.
  - **Database:** Store and retrieve course information, extracted assessment details, review results, and scheduling information.

2.4 User Characteristics

The primary users of the CAS Deadline Management System are CAS administrators and course instructors at McMaster University. Users are expected to have basic computer skills, including navigating web applications and viewing course information. CAS administrators should be familiar with course outline reviews, academic scheduling, and applicable university policies. Course instructors should have knowledge of their own course outlines and assessment schedules. No programming or machine learning knowledge is required. Users should be able to review extracted information, interpret flagged issues and scheduling recommendations, and make informed decisions regarding their courses or administrative responsibilities.

2.5 System Constraints

The CAS Deadline Management System must operate within the technical and institutional constraints of McMaster University. The following constraints apply to the system:

- **Deployment Environment:** The system must be deployable within the computing resources made available by McMaster University. The availability and capacity of these resources may limit the models and processing workloads the system can support.
- **Data Sources:** The system must process course outlines provided in PDF format. Variations in document structure, formatting, and scan quality may affect the accuracy of information extraction.
- **Language Model Cost and Hosting:** The system must use an LLM that is available at no cost to the project. GitHub Copilot should be preferred where it satisfies the system's technical and institutional requirements. Otherwise, the system should prioritize locally hosted, open-weight language models to minimize reliance on external services. Model selection must account for available computing resources and applicable licensing restrictions.
- **Human Decision-Making:** The system must not automatically modify course outlines or assessment dates. All scheduling changes and final compliance decisions remain the responsibility of instructors and authorized administrators.

- **Policy and Accreditation Requirements:** Compliance checks must be based on documented McMaster University policies and applicable CEAB requirements. The system must support human review of flagged issues rather than treating automated results as definitive determinations.

3 Stakeholder Requirements

SR1 Centralized Course Outline Submission: CAS administrators need a centralized location to collect and review course outlines.

SR2 Course Information Extraction: CAS administrators need relevant information extracted from submitted course outlines, including course details, assignments and projects, marking schemes, and midterm dates where available.

SR3 McMaster Policy Review: CAS administrators need support in identifying potential failures of course outlines to comply with McMaster University and CAS department policies.

SR4 CEAB Accreditation Review: CAS administrators need support in reviewing course outlines against CEAB accreditation requirements, including evaluating how each course addresses relevant CEAB graduate attributes.

SR5 Assessment Schedule Analysis: Professors/instructors and CAS administrators need to identify periods with high concentrations of assessment and midterm deadlines across relevant CAS department courses and reassess the schedule when dates are updated.

SR6 Scheduling Recommendations: Professors/instructors need recommendations for alternative midterm and assessment dates that may reduce deadline bottlenecks for students, while being able to make the final decision.

SR7 Instructor Decision-Making: Professors/instructors need to retain authority over whether to accept or reject scheduling recommendations.

SR8 Handling Incomplete Information: Professors/instructors and CAS administrators need to be informed when missing or unreadable course outline information may affect the reliability of compliance reviews or scheduling recommendations.

SR9 Authorized Access: CAS administrators and professors/instructors need access to the information and functions relevant to their roles, while assuring sensitive information is protected and restricted to authorized users.

SR10 Shared Assessment Calendar: Professors/instructors and CAS administrators need access to a shared calendar displaying relevant course assessment dates.

SR11 Exam and Restricted-Period Conflicts: Professors/instructors and CAS administrators need to identify potential conflicts between assessments, exam dates, and applicable restricted periods.

4 User Personas

4.1 Persona 1

- Name: Bob M. Builder (CAS Administrator)
- Description: Bob is responsible for collecting and reviewing CAS course outlines to help ensure they align with McMaster University and CAS department policies, as well as CEAB accreditation requirements. Bob needs an efficient way to carry out his administrative duties while reducing the amount of redundant back and forth communication with instructors to ensure that outlines are aligned with the required policies and accreditation requirements.
- Goals:
  - To collect and review course outlines in one centralized location.
  - To identify missing information and potential policy or accreditation issues.
  - To reduce the time spent manually reviewing multiple course outlines.
  - To reduce unproductive communication time with course instructors.
- Frustrations:
  - Course outlines may use different formats or organize information differently.
  - Manually locating relevant information across many documents can be time-consuming.
  - Missing or unclear information can make it difficult to assess compliance.
- Technical Ability: Moderate (experience with administrative software, document management tools, spreadsheets)

4.2 Persona 2

- Name: Dora Sanchez Explora (Software Engineering Professor)
- Description: Dora teaches Software Engineering Design courses and plans assignments and two midterms for the semester. She wants to provide students with a workload that is easy to manage and deadlines that allow them to learn and adequately prepare for their midterms without overwhelming them in their other courses.

- Goals:
  - To submit and maintain tentative assessment dates efficiently.
  - To identify potential conflicts with assessments in other courses.
  - To receive practical recommendations for distributing assessment deadlines more evenly.
  - To retain final authority over assessment dates for her course.
- Frustrations:
  - Coordinating assessment dates across courses can be difficult.
  - Several courses may schedule major assessments around the same time.
  - Changing an assessment date may create conflicts with other course activities or require additional coordination.
- Technical Ability: Moderate (experience with administrative management systems, online course and university tools)

4.3 Persona 3

- Name: Homer Simpson (4th Year Software Engineering Student)
- Description: Homer is an undergraduate Software Engineering student taking six courses for the Fall term. He is feeling overwhelmed by the workload and is struggling to manage his time between getting his work done and studying for his upcoming back-to-back midterms.
- Goals:
  - To have a more balanced distribution of assessments throughout the term.
  - To avoid periods when multiple major deadlines occur close together.
  - To have sufficient time to prepare for major assessments across different courses.
- Frustrations:
  - Multiple assignments and midterms may fall within the same few days.
  - Heavy assessment periods can make it difficult to allocate study/work time effectively.
  - Assessment dates across different courses may be difficult to coordinate.
- Technical Ability: High (experience with online learning platforms, digital calendars, student productivity tools and software)

4.4 Persona 3

- Name:
- Description:
- Goals:
- Frustrations:
- Technical Ability:

5 Use Cases

5.1 UC1: Submit Course Outline

Primary Actor: CAS Administrator
Preconditions

- The administrator has access to the system.
- A CAS course outline is available for submission.

Main Success Scenario

1. The administrator logs into the system.
2. The administrator selects the option to submit a course outline.
3. The administrator selects and submits the course outline.
4. The system receives the document and attempts to extract relevant course information.
5. The system confirms that the course outline has been submitted successfully.
6. The system makes the submitted outline available for information extraction and review.

Alternative Flows

- If the submitted document cannot be read, the system informs the administrator that a readable document is required.
- If the submission fails, the system informs the administrator and allows them to try again.
- If relevant course information is missing, the system proceeds with the information that is available and identifies the missing information.

- If the Copilot or LLM service is unavailable, the system informs the administrator that automatic information extraction cannot be completed. The submitted course outline remains available, and the administrator can retry extraction when the service becomes available.

Postconditions

- If submission succeeds, the course outline is stored and available for review, even if automatic information extraction could not be completed.
- If submission fails, the administrator is informed that the course outline was not submitted successfully.

5.2 UC2: Review Extracted Course Information

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted successfully.
- The system has attempted to extract relevant course information.

Main Success Scenario

1. The administrator selects a submitted course outline.
2. The system presents the extracted course details and information, including assignments, marking schemes, and midterm dates where available, and flags any missing, uncertain, or ambiguous information for review.
3. The administrator reviews the extracted information against the original course outline.
4. The administrator fills in any missing information or confirms that the information is correct.
5. The system saves the verified course information for later steps and scheduling analysis.

Alternative Flows

- If the system cannot extract certain information, it identifies the missing information for the administrator to review.
- If the system extracts information that is uncertain or ambiguous, it flags the affected information for administrator review and indicates that verification is required.
- If the administrator identifies incorrect information, they correct it before proceeding.

Postconditions

- The extracted course information has been reviewed and corrected where necessary.

5.3 UC3: Review Course Outline Against McMaster and CAS Policies

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- Relevant course information is available for review.
- McMaster University and CAS department policy criteria are available to the system.

Main Success Scenario

1. The administrator selects a course outline for policy review.
2. The system evaluates the available course information against policy criteria.
3. The system identifies potential policy inconsistencies or missing required information.
4. The system presents the findings and supporting information to the administrator.
5. The administrator reviews the findings and determines which issues require further investigation or correction.

Alternative Flows

- If relevant information is missing, the system identifies the limitation and presents any findings that can still be supported.
- If a potential policy issue cannot be determined conclusively, the system flags it for manual review.
- If the necessary policy criteria are unavailable, the system informs the administrator that the review cannot be completed reliably.

Postconditions

- The administrator has access to policy review findings and can identify issues requiring further attention.

5.4 UC4: Review Course Outline Against CEAB Criteria

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- Relevant course information is available for review.
- Applicable CEAB accreditation criteria are available to the system.

Main Success Scenario

1. The administrator selects a course outline for CEAB-related review.
2. The system evaluates the available course information against CEAB criteria.
3. The system identifies potential information gaps that may require further examination.
4. The system presents the findings and supporting information to the administrator.
5. The administrator reviews the findings and determines which issues require further investigation or correction.

Alternative Flows

- If relevant information is missing, the system identifies the limitation and presents any findings that can still be supported.
- If a potential gap requires professional judgment, the system flags it for manual review.
- If the necessary CEAB criteria are unavailable, the system informs the administrator that the review cannot be completed reliably.

Postconditions

- The administrator has access to CEAB-related review findings and can identify issues requiring further attention.

5.5 UC5: Review Course Outline Findings

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- At least one policy or CEAB-related review has been performed.

Main Success Scenario

1. The administrator opens the review findings for a course outline.
2. The system displays identified potential issues, missing information, and relevant supporting details.
3. The administrator examines the findings and determines required follow-up.
4. The administrator uses the findings to support the course outline review process.

Alternative Flows

- If no potential issues are identified, the system indicates that no issues were detected by the review.
- If the findings depend on incomplete or uncertain information, the system identifies those limitations.

Postconditions

- The administrator has reviewed the available findings and identified any issues requiring follow-up.

5.6 UC6: Submit Tentative Assessment Dates

Primary Actor: Professor/Instructor
Preconditions

- The professor or instructor has access to the system.
- The instructor has tentative assessment dates for a relevant course.

Main Success Scenario

1. The instructor selects the relevant course.
2. The instructor enters or submits tentative dates for assessments and midterms.
3. The system checks the submitted information for completeness.
4. The system presents any identified missing or invalid information to the instructor.
5. The instructor corrects any identified issues and submits the information.
6. The system saves the tentative assessment dates for scheduling analysis.

Alternative Flows

- If required assessment information is missing, the system identifies the missing details and asks the instructor to complete them.

- If a date is invalid or cannot be interpreted, the system asks the instructor to correct it.
- If the instructor needs to revise the submitted dates, they can update the information and submit it again.

Postconditions

- The instructor's tentative assessment dates are stored and available for scheduling analysis.

5.7 UC7: Analyze Assessment Schedule

Primary Actor: Professor/Instructor, CAS Administrator
Preconditions

- The user has access to the system.
- Assessment dates are available for the relevant courses, if any have been submitted.

Main Success Scenario

1. The instructor requests an analysis of the assessment schedule.
2. The system gathers the available assessment dates for relevant courses.
3. The system analyzes the distribution of assessments and midterms across the relevant period.
4. The system identifies periods with high concentrations of assessments or potential student workload bottlenecks.
5. The system presents the analysis results to the instructor.

Alternative Flows

- If assessment information is incomplete, the system identifies the missing information and explains how it limits the analysis.
- If insufficient assessment dates are available, the system informs the instructor that a reliable analysis cannot be completed.
- If no significant bottlenecks are identified, the system presents the analysis without recommending unnecessary changes.
- If the instructor or administrator wants to view the assessment calendar without performing an analysis, they can filter the calendar by academic year and program. The system displays the assessment dates matching the selected filters without generating workload analysis or recommendations.

Postconditions

- The instructor or administrator can review the assessment schedule, either through the calendar view or through the results of a schedule analysis, depending on the requested operation.

5.8 UC8: Generate Scheduling Recommendations

Primary Actor: Professor/Instructor
Preconditions

- An assessment schedule analysis has been completed.
- Sufficient information is available to identify potential alternative dates.

Main Success Scenario

1. The instructor requests scheduling recommendations based on the available assessment schedule.
2. The system identifies assessments that contribute to periods with high concentrations of deadlines.
3. The system evaluates potential alternative dates for affected assessments.
4. The system generates recommendations intended to reduce assessment bottlenecks while considering the available scheduling information.
5. The system presents the suggested dates and the available reasoning behind the recommendations.
6. The instructor reviews the recommendations.

Alternative Flows

- If no suitable alternative dates can be identified, the system informs the instructor that it cannot provide a suitable recommendation.
- If relevant scheduling information is missing, the system identifies the limitation.
- If several alternatives are available, the system presents the alternatives for the instructor to evaluate.
- If the LLM/Copilot service goes down, the system notifies the instructor, prevents showing false explanations, and allows retrying once the service recovers.

Postconditions

- Scheduling recommendations (if applicable) are presented to the instructor for consideration. No assessment dates are changed automatically.

5.9 UC9: Review and Decide on Scheduling Recommendations

Primary Actor: Professor/Instructor
Preconditions

- Scheduling recommendations have been generated for the relevant assessment schedule.

Main Success Scenario

1. The instructor opens the scheduling recommendations.
2. The system displays the suggested assessment dates and the rationale for each recommendation.
3. The instructor evaluates the suggestions against course-specific constraints and other relevant considerations.
4. The instructor chooses to accept, reject, or modify a proposed date.
5. The system records the instructor's decision and updates the schedule if an accepted or modified date is submitted.

Alternative Flows

- If the instructor rejects a recommendation, the existing assessment date remains unchanged.
- If the instructor modifies a recommendation, the system allows the revised date to be submitted.
- If the instructor needs more information, they can review the schedule analysis before making a decision.

Postconditions

- The instructor's decision is reflected in the schedule where an update has been submitted. Rejected recommendations do not change the existing assessment dates.

5.10 UC10: Update Assessment Dates and Re-analyze Schedule

Primary Actor: Professor/Instructor
Preconditions

- Assessment dates for the relevant course have previously been submitted.
- The instructor has identified a need to change one or more tentative assessment dates.

Main Success Scenario

1. The instructor selects the relevant course and assessment schedule.
2. The instructor updates one or more tentative assessment dates.
3. The system checks the updated information for completeness and validity, then saves the updated dates.
4. The instructor requests a new assessment schedule analysis.
5. The system analyzes the updated schedule and identifies any remaining or newly introduced bottlenecks.
6. The system presents the updated analysis and any resulting recommendations.

Alternative Flows

- If an updated date is invalid or incomplete, the system asks the instructor to correct it before proceeding.
- If insufficient information is available for re-analysis, the system identifies the missing information.
- If the updated schedule introduces new bottlenecks, the system identifies them in the new analysis.

Postconditions

- The updated tentative assessment dates are available in the system, and a new analysis is available if sufficient information was provided.

6 Functional Requirements

The system shall support the following capabilities, grouped by feature area.

Outline Intake and Extraction

FR1: The system shall accept course outlines in PDF format submitted by a CAS administrator or professor.

FR2: The system shall extract each assessment's name, type, date, and weight from a course outline.

FR3: The system shall extract the learning outcomes and graduate attribute numbers listed in a course outline.

FR4: The system shall flag any extracted field that is missing, including a date that is marked TBD.

FR5: The system shall require a user to review and confirm extracted data before it is used for the calendar, conflict detection, or compliance checking.

FR6: The system shall allow an instructor to enter and update tentative assessment dates for their own courses.

Calendar and Conflict Detection

FR7: The system shall display confirmed and tentative assessment dates in a shared calendar that can be filtered by course, year, and program, with tentative dates marked as such.

FR8: The system shall identify weeks in which a student cohort has more assessments than a configurable threshold.

FR9: The system shall identify assessments scheduled in periods restricted by McMaster policy, such as final exam periods.

FR10: The system shall re-run conflict detection for the affected courses and cohorts whenever an assessment date is added or changed.

Scheduling Suggestions

FR11: The system shall, on explicit request by the instructor, suggest alternative dates, for assessments involved in a detected conflict or listed as TBD.

FR12: The system shall not change any assessment date without the explicit approval of the responsible instructor.

Compliance Checking

FR13: The system shall check each outline against the McMaster course outline policy rules, including required sections and assessment weights totalling 100%, and report each finding.

FR14: The system shall compare the graduate attribute numbers in an outline with the master course sheet and flag any that are missing or inconsistent.

Access and Export

FR15: The system shall require users to log in before they can view or change any data.

FR16: The system shall allow administrators to manage data for all courses, and shall allow instructors to change data only for their own courses.

FR17: The system shall allow an administrator to export extracted assessment data as a CSV file.

7 Data Requirements

7.1 Major Data Entities

- **User:** an administrator or instructor. Both can submit outlines and confirm extracted data. Instructors also enter tentative dates, and request and approve date suggestions, for their own courses.
- **Course:** a CAS course offering in a given term, taught by one instructor. A course may be cross-listed. SFWRENG 4X03, COMPSCI 3X03, and MECHTRON 3X03 are one course taught by one instructor, with a separate course code for each program. The outline and its assessments are stored once.
- **CourseCode:** one listing of a course. Each code belongs to one course, and a course has one or more codes. The program and year level are derived from the code and are not entered separately.
- **CourseOutline:** a submitted course outline PDF. Each outline belongs to one course, including when that course has several course codes.
- **Assessment:** a graded component of a course (midterm, assignment, lab, etc.) with a weight and a date. The date is empty when it is missing or TBD, and an instructor can enter a tentative date.
- **LearningOutcome:** a learning outcome stated in an outline, with any graduate attribute numbers listed beside it. A user reviews each outcome after extraction, and the review status records whether it is still extracted or has been confirmed.
- **MasterCourseRecord:** a row of the master course sheet, listing the graduate attribute numbers expected for a course code. A cross-listed course can match more than one row, one for each of its codes.
- **AcademicCalendarEntry:** a holiday, exam period, or other restricted date range.
- **DateRecommendation:** a suggested alternative date for an assessment, with its explanation and decision.
- **ComplianceFinding:** a problem found when checking an outline against McMaster policy or the master course sheet.

An instructor teaches one or more courses. Each course has one or more course codes and one or more outlines. Each outline contains many assessments and learning outcomes, stored once for the course rather than once per course code. Student cohorts are found by grouping course codes that share the same derived program and year level. Workload conflicts are calculated from the stored assessments when needed, so neither cohorts nor conflicts are stored separately.

7.2 Data Dictionary

Table 2: Data Dictionary

Entity	**Attributes (type): description**
User	user_id (UUID): unique identifier. email (String): user's email address. role (Enum): administrator or instructor.
Course	course_id (UUID): unique identifier. instructor_id (UUID): user who teaches the course. term (String): academic term of the offering.
CourseCode	course_code_id (UUID): unique identifier. course_id (UUID): course this listing belongs to. code (String): course code, e.g., SFWRENG 4X03. program (String, derived): program inferred from the subject in the code, for example Software Engineering from SFWRENG. Not entered separately. year_level (Integer, derived): program year inferred from the first digit of the course number, for example 4 in SFWRENG 4X03 and 3 in COMPSCI 3X03. Not entered separately.
CourseOutline	outline_id (UUID): unique identifier. course_id (UUID): course the outline belongs to. submitted_at (Timestamp): when the outline was submitted.
Assessment	assessment_id (UUID): unique identifier. outline_id (UUID): outline the assessment was extracted from. name (String): name as written in the outline. type (Enum): midterm, assignment, quiz, lab, project, final exam, other. due_date (Date or null): null when missing or TBD. is_tentative (Boolean): true when the date was entered by an instructor and is not final. weight_percent (Decimal): percentage of the final grade. review_status (Enum): extracted or confirmed. source_text (String): outline text the values were extracted from.
LearningOutcome	outcome_id (UUID): unique identifier. outline_id (UUID): outline the outcome appears in. text (String): outcome as written. ga_numbers (List of Integer): graduate attribute numbers listed in the outline. review_status (Enum): extracted or confirmed. source_text (String): outline text the values were extracted from.
MasterCourseRecord	course_code (String): course code this row applies to. Matches a CourseCode. expected_ga_numbers (List of Integer): attribute numbers the master sheet lists for that course code.
AcademicCalendarEntry	start_date (Date): start of the range. end_date (Date): end of the range. restriction (Enum): holiday, exam period, no-assessment period.

Entity	Attributes (type): description
DateRecommendation	recommendation_id (UUID): unique identifier. assessment_id (UUID): assessment the suggestion applies to. suggested_date (Date): proposed date. explanation (String): reason for the suggestion, shown to the instructor. decision (Enum): pending, accepted, rejected.
ComplianceFinding	finding_id (UUID): unique identifier. outline_id (UUID): outline that was checked. message (String): description of the problem found.

7.3 Privacy and Data Retention

DR1: Course outlines and the data extracted from them are generally public. The master course sheet and other department data may be confidential, and shall only be available to authorized CAS users.

DR2: Course and department data shall be processed on CAS-hosted infrastructure. GitHub Copilot is an external service and may be used during development. Only publicly available course outline information may be sent to Copilot. Confidential department data and student personal information shall not be sent to Copilot or to any other external service.

DR3: Users shall only see and change the data their role allows.

DR4: The retention period for outlines, extracted data, and findings shall be 10 years. The Undergraduate Course Management Policy requires that course outlines be retained for at least 10 years.

8 Non-Functional Requirements

NFR1: **Accuracy:**

- The system shall achieve at least 95% accuracy for extracted assessment dates and weights on a representative test dataset of course outlines.
- The system shall achieve at least 95% recall for predefined missing or incomplete information in the same test dataset.
- The system shall identify at least 95% of predefined policy and CEAB compliance issues and report the results in the Verification and Validation Plan.
- These thresholds align with the success measures in the Problem Statement and Goals document.

NFR2: **Usability:**

- The system shall allow the CAS administrators and course instructors described in Section 2.4 to upload course outlines, review extracted information, view assessment schedules, and interpret flagged issues without programming or machine learning knowledge.
- At least 80% of participants in representative usability testing shall complete each designated core task without assistance.
- The 80% target is a project-defined validation threshold intended to demonstrate that the core workflows are usable by the intended non-technical users.

NFR3: **Maintainability:**

- The system shall be organized into modular components for document processing, information extraction, scheduling analysis, and compliance rules.
- Changes to one of these components shall not require unrelated components to be rewritten.
- For representative maintenance tasks defined in the Verification and Validation Plan, the effort required to implement each change shall be documented and assessed against the original implementation effort for the affected component.

NFR4: **Portability:**

- The system shall be deployable in the designated McMaster University hosting environment.
- The system shall be usable through Google Chrome 120 or later, Mozilla Firefox 121 or later, and Safari 17 or later on supported desktop operating systems.
- The deployment and core workflow tests defined in the Verification and Validation Plan shall be executed in each supported browser environment. These browser versions define the project's verification baseline and may be updated for a future release.

NFR5: **Installability and Deployability:**

- A new maintainer shall be able to deploy the system to the designated McMaster University hosting environment by following the documented installation and deployment instructions.
- The documented procedure shall identify required software, configuration values, environment variables, database setup, and service startup steps without relying on undocumented knowledge.
- The deployment shall be verified by a deployment test in the Verification and Validation Plan, including confirmation that the deployed system supports the core workflows.

NFR6: **Performance:**

- Under the test environment and workload defined in the Verification and Validation Plan, the system shall display previously processed assessment calendar data within 3 seconds for at least 95% of requests.
- For a supported course outline of up to 30 pages, the system shall complete document processing and information extraction within 60 seconds for at least 90% of test runs, excluding failures or delays caused by unavailable external dependencies.
- The test environment, document characteristics, workload, and response-time results shall be recorded.
- The 3-second target is a project-defined interactive response target, while the 60-second target bounds processing time for a typical supported outline; both targets shall be validated under the controlled conditions defined in the Verification and Validation Plan.

NFR7: **Reliability:**

- The system shall handle invalid, incomplete, or unsupported input documents without crashing or corrupting previously stored information.
- All predefined invalid-input cases shall produce an appropriate error message, and previously stored records shall remain accessible after a failed upload or processing attempt.
- If the LLM or Copilot service is unavailable, the system shall continue to support viewing and editing stored course and assessment information, viewing the assessment calendar, and exporting available data.
- Submitted course outlines shall be retained, and users shall be able to retry extraction or recommendations when the service becomes available, as described in UC1 and UC8.

NFR8: **Security:**

- The system shall implement the access controls specified in FR15 and FR16, together with the privacy and data-retention requirements in Section 7.3.
- Security verification shall include tests for unauthorized access, role-based permissions, and the protection of sensitive configuration values and credentials.

NFR9: **Accessibility:**

- The system's web interface shall conform to the applicable requirements of the Accessibility for Ontarians with Disabilities Act (AODA) and WCAG 2.0 Level AA.

- Core workflows, including authentication, course-outline submission, information review, calendar viewing, and data export, shall be operable using keyboard navigation and shall provide accessible labels, headings, focus indicators, and error messages.
- Accessibility verification shall include an automated accessibility scan and manual keyboard and screen-reader checks for the core workflows. The applicable WCAG success criteria and test results shall be recorded in the Verification and Validation Plan.

9 AI and Machine Learning Requirements

The system extracts information from course outline PDFs in two steps. First, a Python PDF text extraction library reads the text from each PDF. Then a large language model (LLM) cleans up that text and organizes it into structured data. The LLM also compares the graduate attribute numbers in an outline with the master course sheet. The date suggestions come from a constraint solver (CP-SAT), and the assessment weight check is ordinary deterministic code, so neither uses the LLM. In operation the LLM is hosted on CAS infrastructure and kept behind a replaceable interface, so the model can be changed without changing the rest of the system. During development, GitHub Copilot may be used in its place. Copilot is an external service, and the only model input that may be sent to it is publicly available course outline text (DR2).

9.1 Model Inputs

AIR1: The model shall receive the text that the PDF extraction library produced from each course outline, rather than the PDF file itself, together with the contents of the master course Excel sheet.

AIR2: The operational model shall be hosted on CAS-operated infrastructure. When GitHub Copilot is used during development, only publicly available course outline text may be sent to it (DR2). No student personal information shall be used as model input.

9.2 Model Outputs

AIR3: The model shall organize the extracted outline text into structured data containing, for each assessment, its name, type, date, and weight, and for each learning outcome, its text and the graduate attribute numbers listed beside it (FR2, FR3).

AIR4: Every structured value shall include the passage of extracted text it was taken from, so a reviewer can check it without opening the original document.

AIR5: The model shall only use information present in the extracted text, and shall report a value as missing, including a date that is marked TBD or is too vague to resolve, instead of inventing one (FR4).

AIR6: No model output shall be used for the calendar, conflict detection, or compliance results until a user has confirmed it (FR5).

9.3 Evaluation Metrics

Each metric shall be measured on a set of sample course outlines, using an answer key prepared manually by the team as the reference. Extraction metrics are measured on the final structured output, so they cover the PDF library and the model together.

Table 3: Evaluation Metrics

Metric	**What it measures**	**Target**
Extraction accuracy	Share of assessment dates and weights that are correct in the final structured output (FR2)	At least 90%
Missing-field detection	Share of missing or TBD fields that are flagged (FR4)	At least 90%
Compliance issue detection	Share of known compliance issues that are found (FR13, FR14)	At least 90%

9.4 Known Limitations

The table lists the expected failure modes and how the design contains each one. The main safeguard is that a person confirms every extracted value, with the source text shown beside it. During evaluation, the team also manually reviews a sample of the outputs.

Table 4: Expected Failure Modes

Failure mode	**Effect**	**Mitigation**
PDF library returns messed up or incomplete text (tables, multiple columns, scanned pages)	Dates, weights, or sections are missing or out of order before the model sees them	Flag outlines with little or no extracted text for manual entry; test on outlines with a range of layouts
Model changes or adds values while cleaning up the text	A date or weight that differs from, or appears nowhere in, the outline	Show the source text; reject values with no source passage; require review
Ambiguous dates, such as “week of Oct 20” or “TBD”	A date cannot be resolved without a person	Report the date as missing and never guess

Failure mode	Effect	Mitigation
Reviewer over-trusts the output	Wrong values are confirmed without being checked	Show the source beside each value; confirm explicitly, never by default
Courses missing from the master sheet (technical electives, mechatronics, computer science)	Attribute comparison is not possible	Tell the user the course was not checked, rather than showing no findings
Uneven accuracy across layouts or programs	Some instructors see more errors than others	Test on outlines from each program; add examples from weak areas
CEAB requirements are not exhaustive	A clean result does not mean full accreditation compliance	State that only the encoded rules were checked
LLM unavailable	Extraction and scheduling explanations cannot be completed	Inform the user, keep the submitted outline, do not present an explanation that was not generated, and allow a retry when the service returns (UC_CAS_01, UC_PROF_03)

10 Requirements Traceability Matrices

[The Requirements Traceability Matrix (RTM) is a table that shows the relationship between different types of requirements. It is used to ensure that all requirements are traceable and that no requirements are missed. The typical flow is: Stakeholder Need -¿ Use Case -¿ Requirement —TPLT]

[There is also a traceability matrix between functional and nonfunctional requirements. The purpose of some FRs is to satisfy NFRs. Therefore, it is important to show the relationship between the two. —TPLT]

[Further traceability to test cases and modules will be given in other documents. —TPLT]

Stakeholder Req	Use Case	Functional Req
SR??	UC??	FR??

Table 5: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR??	NFR1

Table 6: Traceability Between Functional and Nonfunctional Requirements

AI Requirement	Related Requirement
AIR1	FR2, FR3
AIR2	DR2
AIR3	FR2, FR3
AIR4	FR5
AIR5	FR4
AIR6	FR5
DR1	FR16
DR3	FR16

Table 7: Traceability Between AI Requirements, Data Requirements, and Functional Requirements

11 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, functional requirements, nonfunctioanl requirements, data requirements, and AI requirements. You don't need a rationale for everything, ust the major decisions that are not obvious. You should have traceability to the entities in the document that you are referencing. —TPLT]

12 Likely Changes

LC1: [Give the likely changes, with traceability to the entity that is likely to change. —TPLT]

13 Unlikely Changes

LC2: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could show the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

14.3 Appendix C: Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

TODO

14.4 Appendix D: Generative AI Disclosure

[Provide full transparency on how generative AI was used in the creation of this document. This includes any text, figures, or tables that were generated by AI. If any skills were created for AI agents to review the documentation, those should be disclosed here. —TPLT]

14.5 Appendix E: Reflection

[This isn't part of the SRS, but it is part of the course. —TPLT]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?